Chapter 03: Essential Python Libraries for Data Science- Part 01

The power of Python in Data Science lies in:

- its **rich ecosystem of specialized libraries**.
- These libraries **reduce the time and effort** required for tasks such as **data cleaning**, **numerical computation**, **visualization**, and **machine learning**.

The goal of this chapter is to **introduce and explore the fundamental Python libraries** that every data scientist should master to work efficiently and effectively.

1. NumPy — *Numerical Python*

1.1. Definition

NumPy (short for *Numerical Python*) is the **foundation of scientific computing** in Python.
It provides high-performance **N-dimensional array objects**, efficient **vectorized operations**, and **broadcasting** capabilities for fast mathematical and statistical computations.

1.2. Why NumPy?

- Fast **numerical computations**
- Handling **multidimensional arrays (ndarray)**
- Performing **vectorized operations** without loops
- foundation of scientific libraries like Pandas, Scikit-learn, and Matplotlib.

In [2]:
```
import numpy as np
# Create an array and perform vectorized computation
a = np.array([1, 2, 3, 4])
b = np.array([5, 6, 7, 8])
result1 = a + b
print(F"result1={result1}")
result2=a*b
print(F"result2={result2}")
```

```
result1=[ 6  8 10 12]
result2=[ 5 12 21 32]
```

In [7]:
```
type(result1)
```

Out[7]:
```
numpy.ndarray
```

1.3. Installation and Import

pip install numpy

import numpy as np

1.4. NumPy — Array Creation

Function	Description	Example	Result
`np.array()`	Create an array from a Python list or tuple	`np.array([1, 2, 3])`	`[1 2 3]`
`np.zeros(shape)`	Create an array filled with **zeros**	`np.zeros((2, 3))`	`[[0. 0. 0.] [0. 0. 0.]]`
`np.ones(shape)`	Create an array filled with **ones**	`np.ones((3, 2))`	`[[1. 1.] [1. 1.] [1. 1.]]`
`np.full(shape, value)`	Create an array filled with a **specific value**	`np.full((2, 2), 7)`	`[[7 7] [7 7]]`
`np.eye(n)`	Create an **identity matrix**	`np.eye(3)`	`[[1. 0. 0.] [0. 1. 0.] [0. 0. 1.]]`
`np.arange(start, stop, step)`	Create a range of evenly spaced **integers**	`np.arange(0, 10, 2)`	`[0 2 4 6 8]`
`np.linspace(start, stop, num)`	Create evenly spaced **floats** between two values	`np.linspace(0, 1, 5)`	`[0. 0.25 0.5 0.75 1.]`
`np.random.rand(d0, d1, …)`	Random floats from **uniform [0, 1)** distribution	`np.random.rand(2, 2)`	e.g. `[[0.42 0.93] [0.18 0.61]]`
`np.random.randn(d0, d1, …)`	Random floats from **normal (Gaussian)** distribution	`np.random.randn(2, 2)`	e.g. `[[-0.54 1.02] [0.33 -1.27]]`
`np.random.randint(low, high, size)`	Random **integers** within a range	`np.random.randint(0, 10, (2, 3))`	e.g. `[[2 9 5] [1 7 0]]`
`np.empty(shape)`	Create an **uninitialized array** (random memory values)	`np.empty((2, 3))`	e.g. `[[1.3e-312 2.5e-312 0.0e+000] [0.0e+000 0.0e+000 0.0e+000]]`
`np.identity(n)`	Create an **n × n identity matrix**	`np.identity(4)`	`[[1. 0. 0. 0.] [0. 1. 0. 0.] [0. 0. 1. 0.] [0. 0. 0. 1.]]`

Function	Description	Example	Result
`np.fromfunction(func, shape)`	Build array from a **function of indices**	`np.fromfunction(lambda i, j: i + j, (3, 3))`	`[[0. 1. 2.] [1. 2. 3.] [2. 3. 4.]]`
`np.fromiter(iterable, dtype)`	Create array from an **iterable object**	`np.fromiter(range(5), dtype=int)`	`[0 1 2 3 4]`

In [1]:
```
import numpy as np
a = np.empty((2,3))
print(a)
```

```
[[2.76886108e-057 1.77296567e+160 7.73423806e-091]
 [1.05890829e+165 8.30730633e-315 0.00000000e+000]]
```

In [3]:
```
import numpy as np
a = np.array([[1, 2, 3, 4],
              [5, 6, 7, 8],
              [9, 10, 11, 12]])
a
```

Out[3]:
```
array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12]])
```

In [4]:
```
np.eye(2,4)
```

Out[4]:
```
array([[1., 0., 0., 0.],
       [0., 1., 0., 0.]])
```

In [5]:
```
np.identity(2,dtype=int)
```

Out[5]:
```
array([[1, 0],
       [0, 1]])
```

In [6]:
```
np.fromfunction(lambda i, j: i * j, (3, 3))
```

Out[6]:
```
array([[0., 0., 0.],
       [0., 1., 2.],
       [0., 2., 4.]])
```

1.5. Array Attributes & Inspection

Functions to **understand array structure**.

Function / Property	Description	Example	Result
`ndim`	Number of dimensions	`a.ndim`	`2`
`shape`	Shape (rows, columns)	`a.shape`	`(3, 4)`
`size`	Total number of elements	`a.size`	`12`
`dtype`	Data type of elements	`a.dtype`	`int64`
`itemsize`	Bytes per element	`a.itemsize`	`8`
`nbytes`	Total bytes of array	`a.nbytes`	`96`

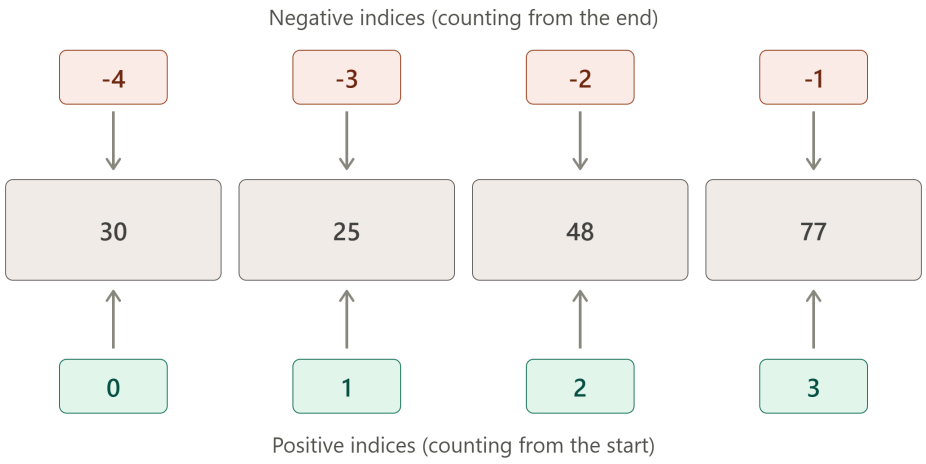
In [8]:
```
a = np.array([[1, 2, 3, 4],
              [5, 6, 7, 8],
              [9, 10, 11, 12]])
print(a.ndim, a.shape, a.size, a.dtype, a.itemsize, a.nbytes)
```

```
2 (3, 4) 12 int64 8 96
```

1.6. Indexing and Slicing

Operation	Description	Example	Result
`a[i]`	Select row *i*	`a[1]`	`[5 6 7 8]`
`a[i, j]`	Select element at row *i*, column *j*	`a[1, 2]`	`7`
`a[i:j]`	Slice rows [start row:end row]	`a[1:3]`	`[[5 6 7 8] [9 10 11 12]]`
`a[:, i]`	Select column *i*	`a[:, 0]`	`[1 5 9]`
`a[::2]`	Step slicing (every 2nd row)	`a[::2]`	`[[1 2 3 4] [9 10 11 12]]`
`a[a > 6]`	Boolean indexing (filter values)	`a[a > 6]`	`[7 8 9 10 11 12]`
`np.where(a > 6)`	Indices where condition is True	`np.where(a > 6)`	`(array([1, 1, 2, 2, 2, 2]), array([2, 3, 0, 1, 2, 3]))`
`a[np.where(a > 6)]`	Elements satisfying condition	`a[np.where(a > 6)]`	`[7 8 9 10 11 12]`
`np.argmax(a)`	Index (flattened) of max element	`np.argmax(a)`	`11`
`np.argmax(a, axis=0)`	Index of max in each column	`np.argmax(a, axis=0)`	`[2 2 2 2]`
`np.argmax(a, axis=1)`	Index of max in each row	`np.argmax(a, axis=1)`	`[3 3 3]`

Operation	Description	Example	Result
`np.argmin(a)`	Index (flattened) of min element	`np.argmin(a)`	`0`
`np.argmin(a, axis=0)`	Index of min in each column	`np.argmin(a, axis=0)`	`[0 0 0 0]`
`np.argmin(a, axis=1)`	Index of min in each row	`np.argmin(a, axis=1)`	`[0 0 0]`
`np.where(a == np.min(a))`	Coordinates of minimum element	`np.where(a == np.min(a))`	`(array([0]), array([0]))`



In [15]:
```
b=np.array([30,25,48,77])
b[2]
```

Out[15]: np.int64(48)

In [19]:
```
print(b[-1:])
```

```
[77]
```

In [17]:
```
b[:-1]
```

Out[17]: array([30, 25, 48])

In [18]:
```
b[::-1]
```

Out[18]: array([77, 48, 25, 30])

In [21]:
```
b[-1::-1]
```

Out[21]: array([77, 48, 25, 30])

In [10]:
```
(a>6)          # creation of is a mask
```

Out[10]:
```
array([[False, False, False, False],
       [False, False,  True,  True],
       [ True,  True,  True,  True]])
```

In [11]:
```
a[a > 6]
```

```
Out[11]: array([ 7,  8,  9, 10, 11, 12])
```

```
In [12]: a[np.where(a > 6)]
```

```
Out[12]: array([ 7,  8,  9, 10, 11, 12])
```

```
In [13]: np.where(a > 6)
```

```
Out[13]: (array([1, 1, 2, 2, 2, 2]), array([2, 3, 0, 1, 2, 3]))
```

```
In [20]: np.where(a > 6, 1, 0)
```

```
Out[20]: array([[0, 0, 0, 0],
                [0, 0, 1, 1],
                [1, 1, 1, 1]])
```

```
In [14]: np.argmax(a)
```

```
Out[14]: np.int64(11)
```

```
In [21]: a[np.where(a == np.argmax(a))]
```

```
Out[21]: array([11])
```

```
In [23]: np.max(a)
```

```
Out[23]: np.int64(12)
```

1.7. Array Operations (Vectorization)

Functions to **perform arithmetic efficiently**

Operation	Description	Example	Result
`a + b`	Element-wise addition	`[1,2,3] + [4,5,6]`	`[5,7,9]`
`a * b`	Element-wise multiplication	`[1,2,3] * [4,5,6]`	`[4,10,18]`
`a * 2`	Element-wise scalar multiplication	`[1,2,3] * 2`	`[2,4,6]`
`a ** 2`	Power	`[2,3,4]**2`	`[4,9,16]`
`np.maximum(a, b)`	Element-wise maximum	`np.maximum([1,5,3], [4,2,6])`	`[4,5,6]`
Broadcasting	Auto expand shapes	`a + 10`	Adds 10 to all elements

1.8. Universal Functions (ufuncs)

Functions to **apply math operations element-wise**.

Function	Description	Example	Result
`np.sqrt(a)`	Square root	`np.sqrt([1,4,9])`	`[1.,2.,3.]`
`np.exp(a)`	Exponential	`np.exp([0,1])`	`[1.,2.718]`
`np.log(a)`	Logarithm	`np.log([1, np.e])`	`[0.,1.]`
`np.sin(a)`	Sine	`np.sin(np.pi/2)`	`1.0`

1.9. Aggregate Functions

Functions to **summarize data**.

Function	Description	Example	Result
`np.sum(a)`	Sum	`np.sum([1,2,3])`	`6`
`np.mean(a)`	Average	`np.mean([1,2,3])`	`2.0`
`np.min(a)`	Minimum	`np.min([4,2,8])`	`2`
`np.max(a)`	Maximum	`np.max([4,2,8])`	`8`
`np.std(a)`	Standard deviation	`np.std([1,2,3])`	`0.816`

In [24]:
```
a = np.array([
    [1,  2,  3,  4],
    [5,  6,  7,  8],
    [9, 10, 11, 12]
])
b = np.array([
    [13, 14, 15, 16],
    [17, 18, 19, 20],
    [21, 22, 23, 24]
])
```

In [26]:
```
np.max(a, axis=1)
```

Out[26]:
```
array([ 4,  8, 12])
```

1.10. Reshaping & Stacking

Functions to **change array shapes** and **combine arrays**.

Function	Description	Example	Result
`reshape`	Change shape	`a.reshape(4,3)`	`[[1 2 3] [4 5 6] [7 8 9] [10 11 12]]`
`ravel`	Flatten array	`a.ravel()`	`[1 2 3 4 5 6 7 8 9 10 11 12]`
`hstack`	Horizontal stack (same rows)	`np.hstack((a,b))`	`[[1 2 3 4 13 14 15 16] [5 6 7 8 17 18 19 20] [9 10 11 12 21 22 23 24]]`

Function	Description	Example	Result
`vstack`	Vertical stack (same columns)	`np.vstack((a,b))`	`[[1 2 3 4] [5 6 7 8] [9 10 11 12] [13 14 15 16] [17 18 19 20] [21 22 23 24]]`
`concatenate`	Stack arrays along axis	`np.concatenate((a,b), axis=0)`	Same as vstack

In [13]:
```
a = np.linspace(0, 1, 6).reshape(3,2)
print(a)
```

```
[[0.  0.2]
 [0.4 0.6]
 [0.8 1. ]]
```

In [14]:
```
a.ravel()
```

Out[14]:
```
array([0. , 0.2, 0.4, 0.6, 0.8, 1. ])
```

In [15]:
```
b=np.array([1,2])
np.vstack((a,b))
```

Out[15]:
```
array([[0. , 0.2],
       [0.4, 0.6],
       [0.8, 1. ],
       [1. , 2. ]])
```

In [16]:
```
np.hstack((b,a.ravel()))
```

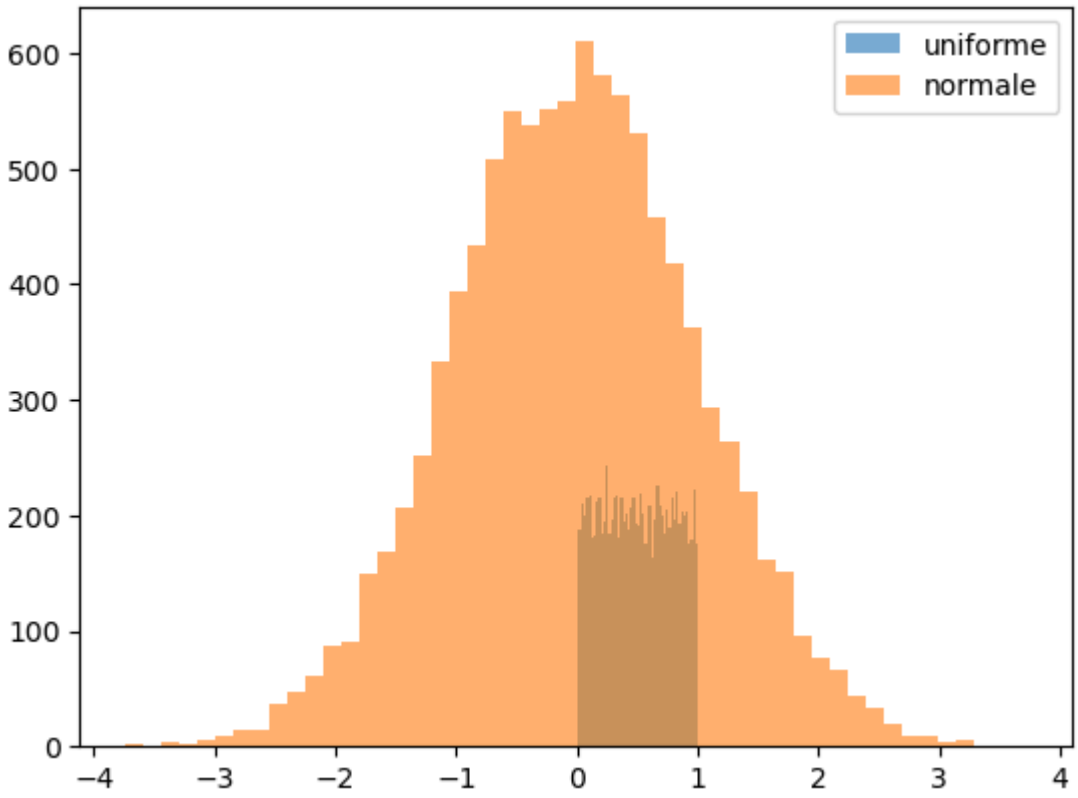
Out[16]:
```
array([1. , 2. , 0. , 0.2, 0.4, 0.6, 0.8, 1. ])
```

1.11. Random Module

Functions to **generate random data**.

Function	Description	Example	Result
`np.random.rand(n)`	Uniform [0,1)	`np.random.rand(3)`	`[0.45, 0.12, 0.88]`
`np.random.randint(a,b)`	Random integers	`np.random.randint(1,10,3)`	`[3,8,1]`
`np.random.randn(n)`	Normal distribution	`np.random.randn(3)`	`[0.3, -0.1, 1.2]`
`np.random.choice(list)`	Random sample	`np.random.choice([1,2,3])`	`2`

In [29]:
```
import matplotlib.pyplot as plt
u = np.random.rand(10000)
n = np.random.randn(10000)
plt.hist(u, bins=50, alpha=0.6, label="uniforme")
plt.hist(n, bins=50, alpha=0.6, label="normale")
plt.legend()
plt.show()
```



In [33]:
```
u = np.random.rand(5,2)
u
```

Out[33]:
```
array([[0.64637837, 0.95324996],
       [0.31388282, 0.96309181],
       [0.40603225, 0.2358386 ],
       [0.14672892, 0.91920822],
       [0.25777257, 0.77819867]])
```

In [36]:
```
v = np.random.randn(10)
v
```

Out[36]:
```
array([-1.04973098,  0.59960725, -0.25516258, -0.50693911, -0.92496703,
        1.27176419, -0.77562124, -0.03592461, -1.00529578,  0.63978273])
```

1.12. NumPy Linear Algebra

NumPy provides a powerful set of functions for performing **linear algebra** operations such as matrix multiplication, solving equations, computing determinants, and eigenvalues.
These tools are essential for numerical analysis, machine learning, and scientific computing.

Function of Linear Algebra

Function	Description	Example
`np.dot(A, B)` or `A @ B`	Matrix or vector multiplication	`np.dot(A, B)`
`A.T`	Transpose of a matrix	`A.T`
`np.linalg.det(A)`	Determinant of matrix `A`	`np.linalg.det(A)`
`np.linalg.inv(A)`	Inverse of matrix `A`	`np.linalg.inv(A)`

Function	Description	Example
`np.linalg.solve(A, b)`	Solves linear system `Ax = b`	`np.linalg.solve(A, b)`
`np.linalg.eig(A)`	Returns eigenvalues and eigenvectors	`np.linalg.eig(A)`
`np.linalg.norm(x)`	Computes vector or matrix norm (magnitude)	`np.linalg.norm(x)`
`np.trace(A)`	Sum of diagonal elements	`np.trace(A)`
`np.linalg.matrix_rank(A)`	Rank of a matrix	`np.linalg.matrix_rank(A)`

Remark:
All advanced linear algebra operations are available in the **`np.linalg`** module, which stands for *NumPy Linear Algebra*.

2. Pandas — *Python Data Analysis Library*

2.1. Definition

Pandas is a high-level library built on top of **NumPy** designed for **data manipulation and analysis**.
It introduces two powerful data structures

- **Series** (1D) and
- **DataFrame** (2D)

which make working with **structured and tabular data** intuitive, fast, and flexible.

2.2. Why Pandas?

- **Easy Data Handling:** Load, clean, transform, and analyze data efficiently
- **Rich I/O Tools:** Import and export data from **CSV**, **Excel**, **SQL**, and **JSON** formats
- **Powerful Indexing:** Label-based selection and alignment
- **Integrated with NumPy:** Enables seamless numerical computation
- **Ideal for DataFrames:** Makes statistical analysis, grouping, and pivoting effortless

2.3. Core Data Structures

Structure	Description	Example
`Series`	1D labeled array	`pd.Series([10, 20, 30], index=['A','B','C'])`
`DataFrame`	2D labeled data table	`pd.DataFrame({'Name':['Amel','Sami'], 'Age':[25,28]})`

2.4. Common Pandas Functions

Category	Key Functions	Description
Data Import/Export	`read_csv()`, `read_excel()`, `read_json()`, `to_csv()`, `to_excel()`	Load and save datasets from/to various formats
Exploration	`head()`, `tail()`, `info()`, `describe()`, `shape`, `dtypes`, `value_counts()`, `unique()`, `nunique()`	View summary, structure, and quick statistics of data
Selection	`loc[]`, `iloc[]`, `at[]`, `iat[]`, `query()`	Access rows and columns by labels, indices, or conditions
Copy & Views	`copy(deep=True)`	Create a deep copy of a DataFrame or Series to avoid modifying the original
Cleaning	`dropna()`, `fillna()`, `replace()`, `duplicated()`, `drop_duplicates()`, `isna()`, `notna()`	Handle missing and duplicate values
Transformation	`apply()`, `map()`, `applymap()`, `astype()`, `rename()`, `replace()`	Transform or convert columns and elements
Computation	`sum()`, `mean()`, `median()`, `min()`, `max()`, `rank()`, `std()`	Perform mathematical and statistical calculations
Aggregation	`groupby()`, `count()`, `agg()`, `transform()`	Aggregate or summarize data by groups
Merging & Joining	`merge()`, `concat()`, `join()`, `combine_first()`	Combine multiple DataFrames
Sorting	`sort_values()`, `sort_index()`	Sort data by one or more columns or by index
Pivot & Reshaping	`pivot()`, `pivot_table()`, `melt()`, `stack()`, `unstack()`	Reshape or reorganize data for analysis
Time Series	`to_datetime()`, `resample()`, `shift()`, `rolling()`, `dt` accessor	Handle, convert, and analyze time-based data

In [31]:
```
###  Example 1
import pandas as pd
# Create a Series
ages = pd.Series([25, 28, 22], name="Age")
print(ages)
print("----")
print(f'{type(ages)} and for elements { ages.dtype}')
```

```
0    25
1    28
2    22
Name: Age, dtype: int64
----
<class 'pandas.core.series.Series'> and for elements int64
```

In [34]:
```
###  Example 2
import numpy as np
age=np.array([25,28,22])
print(age)
```

```
[25 28 22]
```

In [36]:
```
import pandas as pd
ages=pd.Series(age, name="aa", index=['a', 'b','c'])
print(ages)
```

```
a    25
b    28
c    22
Name: aa, dtype: int64
```

In [37]:
```
# Create a DataFrame
data = {'Name': ['Amel', 'Sami', 'Rania'], 'Age': [25, 28, 22]}
df = pd.DataFrame(data, index=['A','B','C'])

print(df)
print("----")
print(type(df))
```

```
    Name  Age
A   Amel   25
B   Sami   28
C  Rania   22
----
<class 'pandas.core.frame.DataFrame'>
```

In [38]:
```
df = pd.DataFrame(data=[['a','b','b'],[22, 13, 15]], index=['A','B'], dtype='string', co

print(df)
print("----")
print(type(df))
```

```
   aa  bb  cc
A   a   b   b
B  22  13  15
----
<class 'pandas.core.frame.DataFrame'>
```

In [39]:
```
df.index.values
```

Out[39]:
```
array(['A', 'B'], dtype=object)
```

In [40]:
```
df=df.reset_index()
df
```

Out[40]:

	index	aa	bb	cc
0	A	a	b	b
1	B	22	13	15

In [41]:
```
df=df.drop(columns="index")
df
```

Out[41]:

	aa	bb	cc
0	a	b	b
1	22	13	15

In [42]:
```
df.columns
```

Out[42]:
```
Index(['aa', 'bb', 'cc'], dtype='object')
```

In [43]:
```
df.index.values
```

Out[43]:
```
array([0, 1])
```

In [44]:
```
df_string = df.select_dtypes(include=['object', 'string'])
df_string
```

Out[44]:

	aa	bb	cc
0	a	b	b
1	22	13	15

In [45]:
```
df_n = df.select_dtypes(include="number").columns.tolist()
df_n
```

Out[45]:
```
[]
```

In [47]:
```
df.columns.tolist()
```

Out[47]:
```
['aa', 'bb', 'cc']
```

2.5. Inspecting Data

Inspecting data is the **first step in data analysis**. It helps to:

- Get a **quick overview** of the data (first/last rows, column names, types).
- Detect **missing values**, **inconsistent types**, or **unexpected formats**.
- Understand **summary statistics** (mean, median, count, etc.) for numerical columns.
- Verify **data dimensions** and ensure DataFrame matches expectations.

In [ ]:
```
```

In [48]:
```
import pandas as pd
import numpy as np
# Create a DataFrame from a dictionary
data = {
    'Name': ['Amel', 'Sami', 'Rania', 'Yacine', 'Nour'],
    'Age': [25, 28, np.nan, 35, 22],
    'City': ['Algiers', 'Oran', 'Constantine', 'Algiers', 'Blida'],
    'Salary': [50000, 54000, 58000, 22450, 48000],
    'JoinDate': ['2020-05-10', '2019-03-15', '2021-07-22', None, '2022-01-11']
}
df = pd.DataFrame(data)
df
```

Out[48]:

	Name	Age	City	Salary	JoinDate
0	Amel	25.0	Algiers	50000	2020-05-10
1	Sami	28.0	Oran	54000	2019-03-15
2	Rania	NaN	Constantine	58000	2021-07-22
3	Yacine	35.0	Algiers	22450	None
4	Nour	22.0	Blida	48000	2022-01-11

In [49]:
```
df.shape              # (rows, columns)
```

Out[49]:
```
(5, 5)
```

In [50]:
```
df.info()             # Overview of columns and data types
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 5 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Name      5 non-null      object
 1   Age       4 non-null      float64
 2   City      5 non-null      object
 3   Salary    5 non-null      int64
 4   JoinDate  4 non-null      object
dtypes: float64(1), int64(1), object(3)
memory usage: 332.0+ bytes
```

In [51]:
```
df.dtypes             # Data types
```

Out[51]:
```
Name         object
Age         float64
City         object
Salary        int64
JoinDate     object
dtype: object
```

In [52]:
```
df.head(3)            # First 3 rows
```

Out[52]:

	Name	Age	City	Salary	JoinDate
0	Amel	25.0	Algiers	50000	2020-05-10
1	Sami	28.0	Oran	54000	2019-03-15
2	Rania	NaN	Constantine	58000	2021-07-22

In [53]:
```
df.tail(2)            # Last 2 rows
```

Out[53]:

	Name	Age	City	Salary	JoinDate
3	Yacine	35.0	Algiers	22450	None
4	Nour	22.0	Blida	48000	2022-01-11

In [54]:
```
df.isnull().sum()
# Count missing values in each column
```

Out[54]:
```
Name        0
Age         1
City        0
Salary      0
JoinDate    1
dtype: int64
```

In [55]:
```
df['City'].unique()
```

Out[55]:
```
array(['Algiers', 'Oran', 'Constantine', 'Blida'], dtype=object)
```

In [56]:
```
df.describe()
```

Out[56]:

	Age	Salary
count	4.000000	5.000000
mean	27.500000	46490.000000
std	5.567764	13976.784323
min	22.000000	22450.000000
25%	24.250000	48000.000000
50%	26.500000	50000.000000
75%	29.750000	54000.000000
max	35.000000	58000.000000

2.6. Selecting & Indexing

Pandas offers several ways to select data from a DataFrame.

- Syntax: DataFrame.loc[row_indexer, column_indexer]
- Use .loc → when working with labels and multiple values/slices.
- Use .iloc → when working with integer positions and multiple values/slices.
- Use .at → when accessing a single value by label (fast).
- Use .iat → when accessing a single value by integer position (fastest).

In [71]:
```
df.index = ["a", "b", "c", "d", "e"]
df
```

Out[71]:

	Name	Age	City	Salary	JoinDate
a	Amel	25.0	Algiers	50000	2020-05-10
b	Sami	28.0	Oran	54000	2019-03-15
c	Rania	NaN	Constantine	58000	2021-07-22
d	Yacine	35.0	Algiers	22450	None
e	Nour	22.0	Blida	48000	2022-01-11

In [72]:
```
df['Name']
```

Out[72]:
```
a      Amel
b      Sami
c     Rania
d    Yacine
e      Nour
Name: Name, dtype: object
```

In [73]:
```
print(df[['Name', 'City']])       # Select multiple columns
```

```
     Name         City
a    Amel      Algiers
b    Sami         Oran
c   Rania  Constantine
d  Yacine      Algiers
e    Nour        Blida
```

In [76]:
```
print(df.loc['b ':])       # Select a row
```

```
     Name   Age         City  Salary    JoinDate
c   Rania   NaN  Constantine   58000  2021-07-22
d  Yacine  35.0      Algiers   22450        None
e    Nour  22.0        Blida   48000  2022-01-11
```

In [78]:
```
print(df.loc[:'d'])
```

```
     Name   Age         City  Salary    JoinDate
a    Amel  25.0      Algiers   50000  2020-05-10
b    Sami  28.0         Oran   54000  2019-03-15
c   Rania   NaN  Constantine   58000  2021-07-22
d  Yacine  35.0      Algiers   22450        None
```

In [82]:
```
print(df.iloc[2])
```

```
Name              Rania
Age                 NaN
City        Constantine
Salary            58000
JoinDate     2021-07-22
Name: c, dtype: object
```

In [59]:
```
print(df.iloc[0,:])
```

```
Name              Amel
Age               25.0
City           Algiers
Salary           50000
JoinDate    2020-05-10
Name: 0, dtype: object
```

In [60]:
```
print(df.loc[:, "Age"])       # Select a column
```

```
0    25.0
1    28.0
2     NaN
3    35.0
4    22.0
Name: Age, dtype: float64
```

In [61]:
```
print(df.iloc[:, 1])       # Select a row
```

```
0    25.0
1    28.0
2     NaN
3    35.0
4    22.0
Name: Age, dtype: float64
```

In [42]:
```
print(df.loc[0:2, ['Name', 'Age']])  # By labels
```

```
    Name   Age
0   Amel  25.0
1   Sami  28.0
2  Rania   NaN
```

In [43]:
```
print(df.iloc[0:2, 0:1] )           # By integer positions
```

```
   Name
0  Amel
1  Sami
```

In [44]:
```
print(df.at[2, 'Age'])              # Fast scalar access
```

```
nan
```

In [45]:
```
print(df.iat[1, 1])           # Fast Position-based, single scalar
```

```
28.0
```

2.7. Filtering Rows

Pandas allows to filter rows based on conditions.

- Use & for AND, | for OR in multiple conditions.
- Parentheses () are required around each condition when using & or |.
- .query() allows a string-based syntax that can be easier to read for complex filters.
- `.where()` keeps the same shape as the original DataFrame, **replacing non-matching values with NaN** instead of dropping rows.

In [46]:
```
# Filter rows where Age > 25
print(df['Age']>25)
```

```
0    False
1     True
2    False
3     True
4    False
Name: Age, dtype: bool
```

In [47]:
```
print(df[df['Age'] > 25])
```

```
     Name   Age     City  Salary    JoinDate
1    Sami  28.0     Oran   54000  2019-03-15
3  Yacine  35.0  Algiers   22450        None
```

In [48]:
```
# Filter rows with multiple conditions (Age > 25 AND City == 'Algiers')
df[(df['Age'] > 25) & (df['City'] == 'Algiers')]
```

Out[48]:

	Name	Age	City	Salary	JoinDate
3	Yacine	35.0	Algiers	22450	None

In [49]:
```
# Using query syntax for readability
df.query('Salary > 50000 and City == "Oran"')
```

Out[49]:

	Name	Age	City	Salary	JoinDate
1	Sami	28.0	Oran	54000	2019-03-15

In [50]:
```
# Using where()
df_where = df.where(df["Age"] > 30)   # Non-matching rows become NaN
print(df_where)
```

```
     Name   Age     City   Salary JoinDate
0     NaN   NaN      NaN      NaN      NaN
1     NaN   NaN      NaN      NaN      NaN
2     NaN   NaN      NaN      NaN      NaN
3  Yacine  35.0  Algiers  22450.0     None
4     NaN   NaN      NaN      NaN      NaN
```

In [51]:
```
df_where.dropna(subset=["Age"], inplace=True)  # Optionally remove NaN
print(df_where)
```

```
     Name   Age     City   Salary JoinDate
3  Yacine  35.0  Algiers  22450.0     None
```

2.8. Adding, Renaming & Deleting Columns

Pandas allows to easily **add, rename, and delete columns** in a DataFrame.

- After these operations, use df.head() to quickly verify changes.
- Use inplace=True to apply changes directly to the DataFrame.

In [52]:
```
# Adding, Renaming & Deleting Columns
df['Bonus'] = df['Salary'] * 0.10                # New column
df.rename(columns={'Salary': 'MonthlySalary'}, inplace=True)
df.drop(columns='Bonus', inplace=True)             # Remove column
```

In [53]:
```
df.head()
```

Out[53]:

	Name	Age	City	MonthlySalary	JoinDate
0	Amel	25.0	Algiers	50000	2020-05-10
1	Sami	28.0	Oran	54000	2019-03-15
2	Rania	NaN	Constantine	58000	2021-07-22
3	Yacine	35.0	Algiers	22450	None
4	Nour	22.0	Blida	48000	2022-01-11

2.9. Data transformation

Data transformation is an essential step to **prepare, clean, and manipulate data** before analysis.

- Transformation should be done after inspection (checking for missing values, data types, outliers).
- Chain transformations carefully to maintain clean and readable code.

In [54]:
```
# create a new copy of a df
df1 = df.copy()
# drop missing values
df.dropna(inplace=True)
```

In [55]:
```
print(df)
print("----------")
print(df1)
```

```
   Name   Age     City  MonthlySalary    JoinDate
0  Amel  25.0  Algiers          50000  2020-05-10
1  Sami  28.0     Oran          54000  2019-03-15
4  Nour  22.0    Blida          48000  2022-01-11
----------
     Name   Age         City  MonthlySalary    JoinDate
0    Amel  25.0      Algiers          50000  2020-05-10
1    Sami  28.0         Oran          54000  2019-03-15
2   Rania   NaN  Constantine          58000  2021-07-22
3  Yacine  35.0      Algiers          22450        None
4    Nour  22.0        Blida          48000  2022-01-11
```

In [56]:
```
# --- String Transformation ---
# Convert all names to uppercase
df['Name'] = df['Name'].str.upper()
print(df)
```

```
   Name   Age     City  MonthlySalary    JoinDate
0  AMEL  25.0  Algiers          50000  2020-05-10
1  SAMI  28.0     Oran          54000  2019-03-15
4  NOUR  22.0    Blida          48000  2022-01-11
```

In [57]:
```
# --- Replacing Values ---
# Replace 'Blida' with 'Blida City' in the 'City' column
df['City'] = df['City'].replace('Blida', 'Blida City')
print(df)
```

```
   Name   Age        City  MonthlySalary    JoinDate
0  AMEL  25.0     Algiers          50000  2020-05-10
1  SAMI  28.0        Oran          54000  2019-03-15
4  NOUR  22.0  Blida City          48000  2022-01-11
```

In [58]:
```python
# --- Transforming Values in a Column ---
# Convert all names in the 'Name' column to lowercase using the map() function.
# map() applies the given function (lambda x: x.lower()) to each element of the Series.
df['Name'] = df['Name'].apply(lambda x: x.lower())
print(df)
```

```
   Name   Age        City  MonthlySalary    JoinDate
0  amel  25.0     Algiers          50000  2020-05-10
1  sami  28.0        Oran          54000  2019-03-15
4  nour  22.0  Blida City          48000  2022-01-11
```

In [59]:
```python
# --- Applying Functions ---
# Create a new column 'AgeGroup' using apply() + lambda
# (apply() is more flexible and can use multiple columns if needed)
df['AgeGroup'] = df['Age'].apply(lambda x: 'Young' if x < 30 else 'Senior')
print(df)
```

```
   Name   Age        City  MonthlySalary    JoinDate AgeGroup
0  amel  25.0     Algiers          50000  2020-05-10    Young
1  sami  28.0        Oran          54000  2019-03-15    Young
4  nour  22.0  Blida City          48000  2022-01-11    Young
```

In [60]:
```python
# --- Changing Data Types ---
# Convert MonthlySalary from float to int
df['MonthlySalary'] = df['MonthlySalary'].astype(int)
print(df)
```

```
   Name   Age        City  MonthlySalary    JoinDate AgeGroup
0  amel  25.0     Algiers          50000  2020-05-10    Young
1  sami  28.0        Oran          54000  2019-03-15    Young
4  nour  22.0  Blida City          48000  2022-01-11    Young
```

In [61]:
```python
# Define a custom function
def simplify(x):
    if x == "Young":
        return "Y"
    elif x == "Senior":
        return "S"
    else:
        return x

# Apply the function to the 'Age' column
df["Age_Short"] = df["Age"].apply(simplify)
df
```

Out[61]:

	Name	Age	City	MonthlySalary	JoinDate	AgeGroup	Age_Short
0	amel	25.0	Algiers	50000	2020-05-10	Young	25.0
1	sami	28.0	Oran	54000	2019-03-15	Young	28.0
4	nour	22.0	Blida City	48000	2022-01-11	Young	22.0

2.10. Sorting & Ranking

- **sort_values()**: Reorders the rows in the DataFrame based on a column's values.

- **rank()**: Assigns a ranking number to each value but keeps the original row order.

In [62]:
```
df = pd.DataFrame({
    'Name': ['Amel', 'Sami', 'Rania', 'Yacine', 'Nour'],
    'Age': [25, 28, 28, 35, 22],
    'City': ['Algiers', 'Oran', 'Constantine', 'Algiers', 'Blida'],
    'Salary': [50000, 54000, 58000, 22450, 48000],
    'JoinDate': ['2020-05-10', '2019-03-15', '2021-07-22', '2022-01-11', '2022-01-11']
})
df
```

Out[62]:

	Name	Age	City	Salary	JoinDate
0	Amel	25	Algiers	50000	2020-05-10
1	Sami	28	Oran	54000	2019-03-15
2	Rania	28	Constantine	58000	2021-07-22
3	Yacine	35	Algiers	22450	2022-01-11
4	Nour	22	Blida	48000	2022-01-11

In [63]:
```
df.sort_values(by='Salary', ascending=True, inplace=True)
df
```

Out[63]:

	Name	Age	City	Salary	JoinDate
3	Yacine	35	Algiers	22450	2022-01-11
4	Nour	22	Blida	48000	2022-01-11
0	Amel	25	Algiers	50000	2020-05-10
1	Sami	28	Oran	54000	2019-03-15
2	Rania	28	Constantine	58000	2021-07-22

In [64]:
```
df['DateRank'] = df['JoinDate'].rank(ascending=True)
df
```

Out[64]:

	Name	Age	City	Salary	JoinDate	DateRank
3	Yacine	35	Algiers	22450	2022-01-11	4.5
4	Nour	22	Blida	48000	2022-01-11	4.5
0	Amel	25	Algiers	50000	2020-05-10	2.0
1	Sami	28	Oran	54000	2019-03-15	1.0
2	Rania	28	Constantine	58000	2021-07-22	3.0

2.11. Grouping & Aggregation

allow to split data into groups based on specific columns and apply summary functions (like mean, sum, or count) to each group.

In [65]:
```
grouped = df.groupby('Age').sum()
grouped
```

Out[65]:

Age	Name	City	Salary	JoinDate	DateRank
22	Nour	Blida	48000	2022-01-11	4.5
25	Amel	Algiers	50000	2020-05-10	2.0
28	SamiRania	OranConstantine	112000	2019-03-152021-07-22	4.0
35	Yacine	Algiers	22450	2022-01-11	4.5

In [66]:
```
grouped = df.groupby('City').agg({
    'Salary': ['mean', 'max', 'min'],
    'Age': 'mean'
})
grouped
```

Out[66]:

	Salary			Age
	mean	max	min	mean
City				
Algiers	36225.0	50000	22450	30.0
Blida	48000.0	48000	48000	22.0
Constantine	58000.0	58000	58000	28.0
Oran	54000.0	54000	54000	28.0

2.12. Merging & Concatenation

- **pd.merge()** is used to combine two DataFrames based on a column
- **pd.concatenate()** is used to concatenate horizontaly/vertically | Operation | Purpose | Joins By | Example | | ---------- | ---------- | ---------- | ---------- | | `merge()` | Combine DataFrames by **common column(s)** | Column values | `pd.merge(df, dept, on='City')` | | `concat()` | Combine DataFrames **by rows or columns** | Index position | `pd.concat([df, df2])` |

In [67]:
```
# Second DataFrame
dept = pd.DataFrame({
    'City': ['Algiers', 'Oran', 'Constantine', 'Blida City'],
    'Department': ['HR', 'IT', 'Finance', 'Marketing']
})

# Merge on common column
merged = pd.merge(df, dept, on='City', how='left') # how: left, inner, right, outer
merged
```

Out[67]:

	Name	Age	City	Salary	JoinDate	DateRank	Department
0	Yacine	35	Algiers	22450	2022-01-11	4.5	HR
1	Nour	22	Blida	48000	2022-01-11	4.5	NaN
2	Amel	25	Algiers	50000	2020-05-10	2.0	HR
3	Sami	28	Oran	54000	2019-03-15	1.0	IT
4	Rania	28	Constantine	58000	2021-07-22	3.0	Finance

In [68]:
```
#Concatenate vertically (one below another)
df2 = pd.DataFrame({
    'Name': ['Hana'],
    'City': ['Setif'],
    'Salary': [4800]
})

# Stack rows (same columns)
concatenated = pd.concat([df, df2], ignore_index=True)
print("\n---- CONCATENATED (Vertical) ----")
print(concatenated)
```

```
---- CONCATENATED (Vertical) ----
     Name   Age         City  Salary    JoinDate  DateRank
0  Yacine  35.0      Algiers   22450  2022-01-11       4.5
1    Nour  22.0        Blida   48000  2022-01-11       4.5
2    Amel  25.0      Algiers   50000  2020-05-10       2.0
3    Sami  28.0         Oran   54000  2019-03-15       1.0
4   Rania  28.0  Constantine   58000  2021-07-22       3.0
5    Hana   NaN        Setif    4800         NaN       NaN
```

In [69]:
```
# Combine DataFrames column-wise (axis=1), Horizontaly
concat_side = pd.concat([df, df2], axis=1)
print("\n---- CONCATENATED (Horizontal) ----")
print(concat_side)
```

```
---- CONCATENATED (Horizontal) ----
     Name  Age         City  Salary    JoinDate  DateRank  Name   City  Salary
3  Yacine   35      Algiers   22450  2022-01-11       4.5   NaN    NaN     NaN
4    Nour   22        Blida   48000  2022-01-11       4.5   NaN    NaN     NaN
0    Amel   25      Algiers   50000  2020-05-10       2.0  Hana  Setif  4800.0
1    Sami   28         Oran   54000  2019-03-15       1.0   NaN    NaN     NaN
2   Rania   28  Constantine   58000  2021-07-22       3.0   NaN    NaN     NaN
```

2.13. Pivot Tables & Reshaping

Function	Category	Purpose	Key Parameters	What It Does	When to Use
`pivot()`	Reshaping	Reorganize data (wide format)	`index`, `columns`, `values`	Creates a new table by turning unique combinations of `index` and `columns` into cells.	When your data has **unique index/column pairs** and you just want to reshape (no aggregation).

Function	Category	Purpose	Key Parameters	What It Does	When to Use
`pivot_table()`	Summarization	Aggregate and reshape data	`index`, `columns`, `values`, `aggfunc`	Works like `pivot()`, but handles **duplicates** by applying an **aggregation** (default = `mean`).	When your data has **duplicates** or you need to **summarize** numeric data.
`melt()`	Reshaping	Unpivot data (long format)	`id_vars`, `value_vars`, `var_name`, `value_name`	Converts columns into rows, creating a long and tidy structure.	When you need to **prepare data for plotting, modeling, or normalization**.

In [70]:
```
df = pd.read_csv('sample_10_rows.csv')
print(df)
```

```
     Name   Age         City  Salary    JoinDate
0    Amel  25.0      Algiers   50000  2020-05-10
1    Sami  28.0         Oran   54000  2019-03-15
2   Rania   NaN  Constantine   58000  2021-07-22
3  Yacine  35.0      Algiers   22450         NaN
4    Nour  22.0        Blida   48000  2022-01-11
5   Farid  40.0        Setif   60000  2018-09-03
6    Lina  27.0         Oran   52000  2020-11-25
7   Karim  30.0        Blida   45000  2021-02-14
8   Sofia   NaN      Algiers   47000         NaN
9   Walid  33.0  Constantine   53000  2019-12-30
```

a) `pivot()` — Reshaping Data in Pandas

`pivot()` creates a **new DataFrame** where one column becomes the **index**, another defines the **columns**, and a third provides the **cell values**.

Note: Each combination of `index` / `columns` must be **unique** — otherwise, use `pivot_table()`.

General Syntax

new_df = df.pivot(index='col1', columns='col2', values='col3')

- **col1** → becomes the row index of the new table
- **col2** → becomes the new columns
- **col3** → fills the cell values

In [71]:
```
df1 = df.drop_duplicates(subset="City", keep="first")
print(df1)
```

```
    Name   Age         City  Salary    JoinDate
0   Amel  25.0      Algiers   50000  2020-05-10
1   Sami  28.0         Oran   54000  2019-03-15
2  Rania   NaN  Constantine   58000  2021-07-22
4   Nour  22.0        Blida   48000  2022-01-11
5  Farid  40.0        Setif   60000  2018-09-03
```

In [72]:
```
pivot_df = df1.pivot(index='City', columns='JoinDate', values='Salary')
print("\nPivot result:")
print(pivot_df)
```

```
Pivot result:
JoinDate     2018-09-03  2019-03-15  2020-05-10  2021-07-22  2022-01-11
City
Algiers             NaN         NaN     50000.0         NaN         NaN
Blida               NaN         NaN         NaN         NaN     48000.0
Constantine         NaN         NaN         NaN     58000.0         NaN
Oran                NaN     54000.0         NaN         NaN         NaN
Setif           60000.0         NaN         NaN         NaN         NaN
```

b) `pivot_table()` — Creating Summarized Tables in Pandas

`pivot_table()` works like `pivot()`, but it allows **duplicate combinations** of index/columns by applying an **aggregation function** (like `mean`, `sum`, etc.).

Note: It's more flexible than `pivot()` because it can handle repeated values.

General Syntax

new_df = df.pivot_table(

```
    index='col1',

    columns='col2',

    values='col3',

    aggfunc='mean'   # default aggregation is 'mean'
```

)

In [73]:
```
pivot_table = df.pivot_table(
    index='City',      # become the index
    values='Age',  # column to summarize
    aggfunc='mean'     # aggregate duplicates using mean
)
print(pivot_table)
```

```
             Age
City
Algiers      30.0
Blida        26.0
Constantine  33.0
Oran         27.5
Setif        40.0
```

c) `melt()` — Unpivoting (Reshaping) Data in Pandas

`melt()` performs the **opposite of** `pivot()`:
it **turns columns into rows**, making the data **longer** instead of wider.

It's useful for **normalizing** or **preparing data for analysis and visualization**.

General Syntax

new_df = df.melt(

```
    id_vars=['col1'],         # columns to keep as identifiers

    value_vars=['col2', 'col3'],  # columns to unpivot

    var_name='variable',      # name for the new column holding former
    column names

    value_name='value'        # name for the new column holding values
```

)

In [74]:
```
# unpivot df
df_melted = df.melt(
    id_vars=["Name", "City"],          # columns to keep fixed
    value_vars=["Age", "Salary", ],  # columns to unpivot
    var_name="resume",              # name of the new column for former column names
    value_name="Value"                 # name of the new column for the data
)

print(df_melted)
```

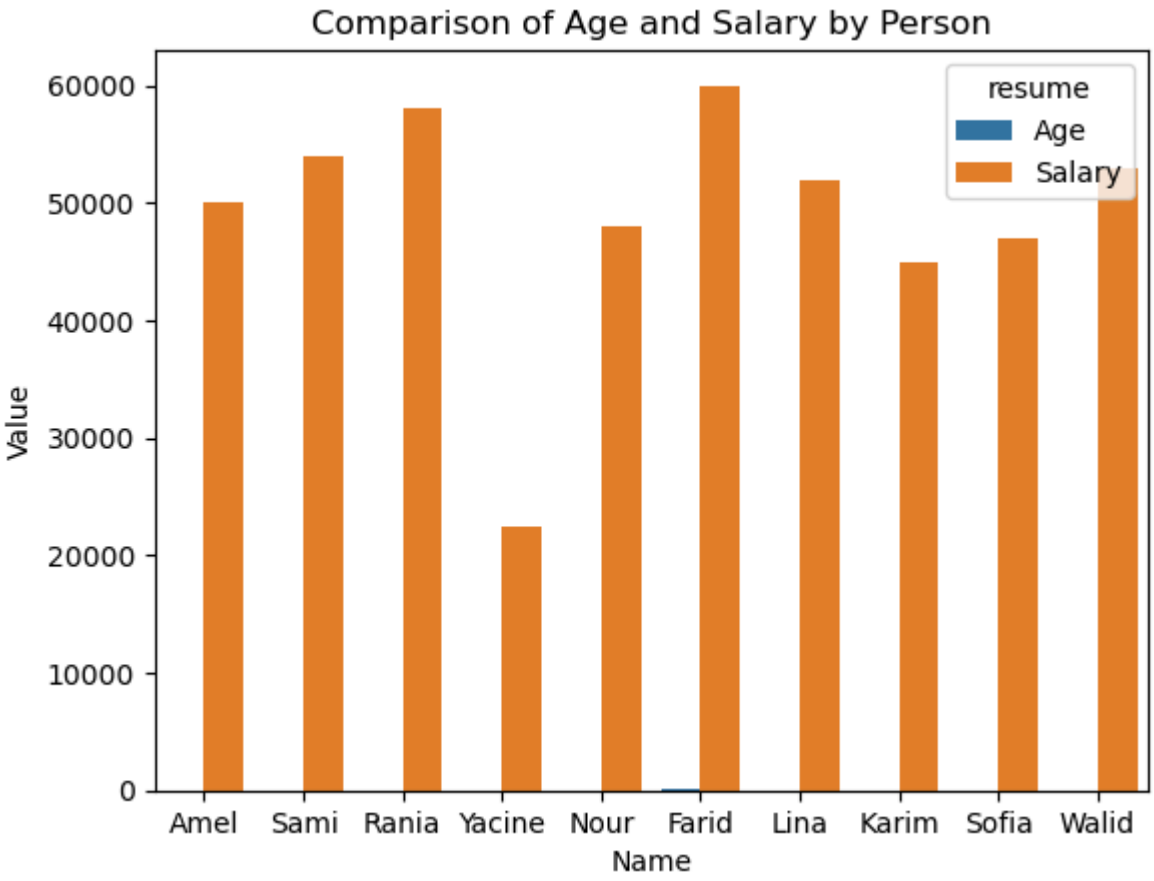
```
      Name         City  resume    Value
0     Amel      Algiers     Age     25.0
1     Sami         Oran     Age     28.0
2    Rania  Constantine     Age      NaN
3   Yacine      Algiers     Age     35.0
4     Nour        Blida     Age     22.0
5    Farid        Setif     Age     40.0
6     Lina         Oran     Age     27.0
7    Karim        Blida     Age     30.0
8    Sofia      Algiers     Age      NaN
9    Walid  Constantine     Age     33.0
10    Amel      Algiers  Salary  50000.0
11    Sami         Oran  Salary  54000.0
12   Rania  Constantine  Salary  58000.0
13  Yacine      Algiers  Salary  22450.0
14    Nour        Blida  Salary  48000.0
15   Farid        Setif  Salary  60000.0
16    Lina         Oran  Salary  52000.0
17   Karim        Blida  Salary  45000.0
18   Sofia      Algiers  Salary  47000.0
19   Walid  Constantine  Salary  53000.0
```

In [75]:
```
import seaborn as sns
import matplotlib.pyplot as plt

sns.barplot(
    data=df_melted,
    x="Name",
    y="Value",
```

```
    hue="resume"  # différencie Age et Salary
)

plt.title("Comparison of Age and Salary by Person")
plt.show()
```



Age doesn't appear in the bar plot, why?

2.14. Time Series in Pandas

A time series is a sequence of data points indexed by time (daily, monthly, yearly...).

Pandas provides powerful tools for analyzing and transforming time-based data.

In [76]:
```
data = {
    "Name": ["nour", "amel", "sami"],
    "JoinDate": ["2022-01-11", "2020-05-10", "2019-03-15"],
}

df = pd.DataFrame(data)
print(df.dtypes)

df["JoinDate"] = pd.to_datetime(df["JoinDate"])
print("-----")
print(df)
print("-----")
# Vérifier le type
print(df.dtypes)
```

```
Name        object
JoinDate    object
dtype: object
-----
   Name   JoinDate
0  nour 2022-01-11
1  amel 2020-05-10
2  sami 2019-03-15
-----
Name                 object
JoinDate    datetime64[ns]
dtype: object
```

2.15. Exporting Data

Pandas provides simple functions to **save DataFrames** into different file formats such as **CSV, Excel, JSON, or SQL**.

Common Export Functions

Function	Description	Example
`df.to_csv('file.csv', index=False)`	Export to **CSV** (comma-separated values)	Saves the DataFrame as a CSV file
`df.to_excel('file.xlsx', index=False)`	Export to **Excel**	Requires the `openpyxl` package
`df.to_json('file.json')`	Export to **JSON** format	Good for web data exchange
`df.to_html('file.html')`	Export to **HTML table**	Can be displayed in a browser
`df.to_sql('table_name', conn)`	Write to a **SQL database**	Needs a valid database connection
`df.to_pickle('file.pkl')`	Save in **binary format** (fast read/write)	For internal Python use only

Common Parameters

- `index=False` → do not include the index column in the output
- `sep=';'` → change the separator for CSV files
- `header=False` → skip column names
- `encoding='utf-8'` → set text encoding

In [77]:
```
# Exporting Data
merged.to_csv('employees_cleaned.csv', index=False)
```

Takeaway:

Pandas is a Swiss-army knife for data analysis

- from loading data to cleaning, transforming, aggregating, and visualizing it
- all in just a few lines of code.